



SR University, Ananthasagar, Warangal, Telangana-506371

MACHINE LEARNING PROJECT

Title of the Project

Linked in software engineering jobs dataset

Submitted by

SIDDAM SHETTI KAVYA

Roll No: 2403A51037

Batch No: 02

B.Tech 2nd Year

Under the Guidance of

Dr. K.VENKATESWARA REDDY

Department of CSE SR University



Linked in software engineering jobs dataset



Linked in software engineering jobs dataset

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
df = pd.read_csv('/content/postings[1].csv')
print("Dataset loaded:")
print(df.head())

# Create Skills_Count column (number of skills per job)
df['Skills_Count'] = df['job_skills'].apply(
    lambda x: len(str(x).split(',')) if pd.notna(x) else 0
)

# Encode categorical columns (except numeric columns)
for col in df.select_dtypes(include='object').columns:
    if col not in ['Salary', 'Skills_Count']:
        df[col] = LabelEncoder().fit_transform(df[col].astype(str))

# Fill missing values with 0
df.fillna(0, inplace=True)

print("Data preprocessing complete.")
```

... Dataset loaded:

	job_title	company \
0	C# Software Engineer	E Tech Group
1	Software Implementation Engineer	Kardex
2	Senior Software Engineer, Back End (Go, AWS, J...	Jobs for Humanity
3	Senior Manager, Software Engineering, Full Stack	Jobs for Humanity
4	Lead Software Engineer, Full Stack(JavaScript/...	Jobs for Humanity

	job_location	job_link \
0	West Chester, OH	https://www.linkedin.com/jobs/view/c%23-softwa...
1	Cincinnati, OH	https://www.linkedin.com/jobs/view/software-im...
2	Chattanooga, TN	https://www.linkedin.com/jobs/view/senior-soft...

...	first_seen	search_city	search_country	job_level	job_type	\
0	2023-12-25	Covington	United States	Associate	Remote	
1	2023-12-25	Covington	United States	Associate	Remote	
2	2023-12-25	Chattanooga	United States	Mid senior	Onsite	
3	2023-12-25	Chattanooga	United States	Mid senior	Onsite	
4	2023-12-25	Chattanooga	United States	Mid senior	Onsite	
						job_summary \
0	At E Tech Group, joining our team means joinin...					
1	Kardex Remstar is looking for a\nSoftware Impl...					
2	Company Description\nJobs for Humanity is part...					
3	Company Description\nJobs for Humanity is part...					
4	Company Description\nJobs for Humanity is part...					
						job_skills
0	C#, .NET, WPF, ASP.NET MVC, WebAPI, C++, Progr...					
1	Software Implementation, Software Testing, Sof...					
2	Java, Python, SQL, Node.js, Go, Scala, AWS, GC...					
3	JavaScript, Java, TypeScript, SQL, Python, Go,...					
4	JavaScript, Java, AWS, Vue.js, HTML/CSS, TypeS...					



linear regression

```
# Regression dataset
X_reg = df.drop(['Skills_Count'], axis=1)
y_reg = df['Skills_Count']

# Train-test split
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

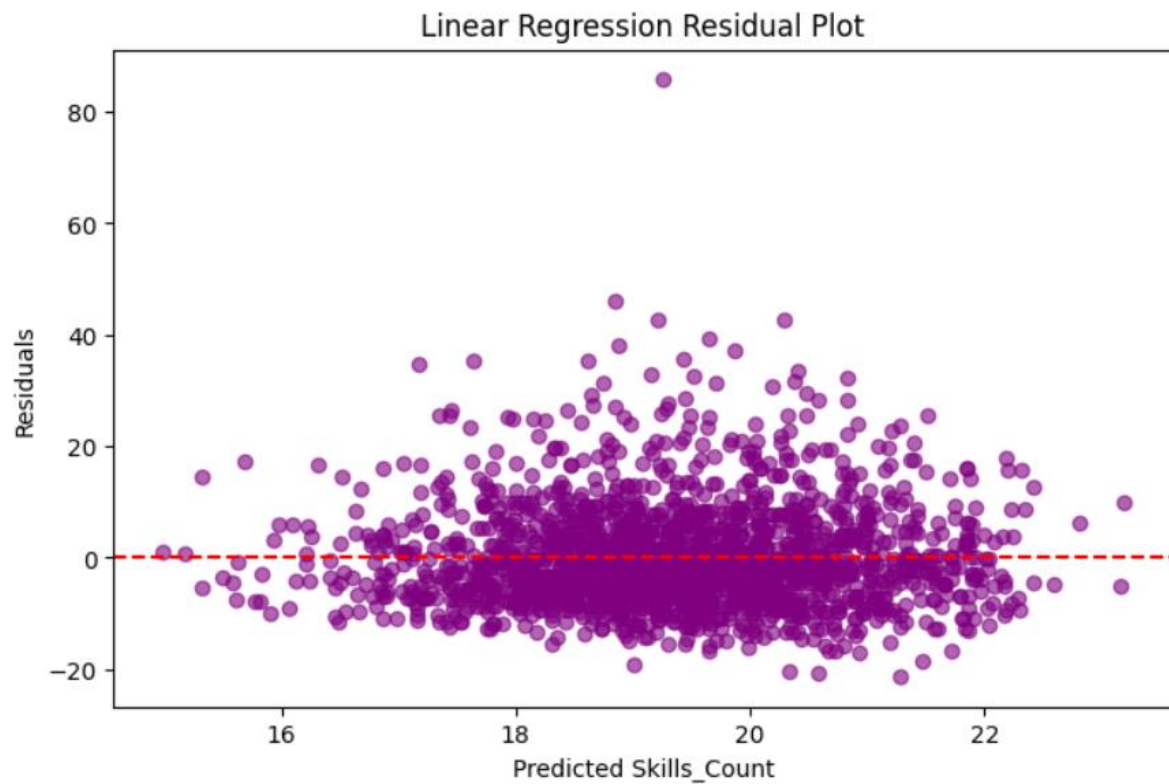
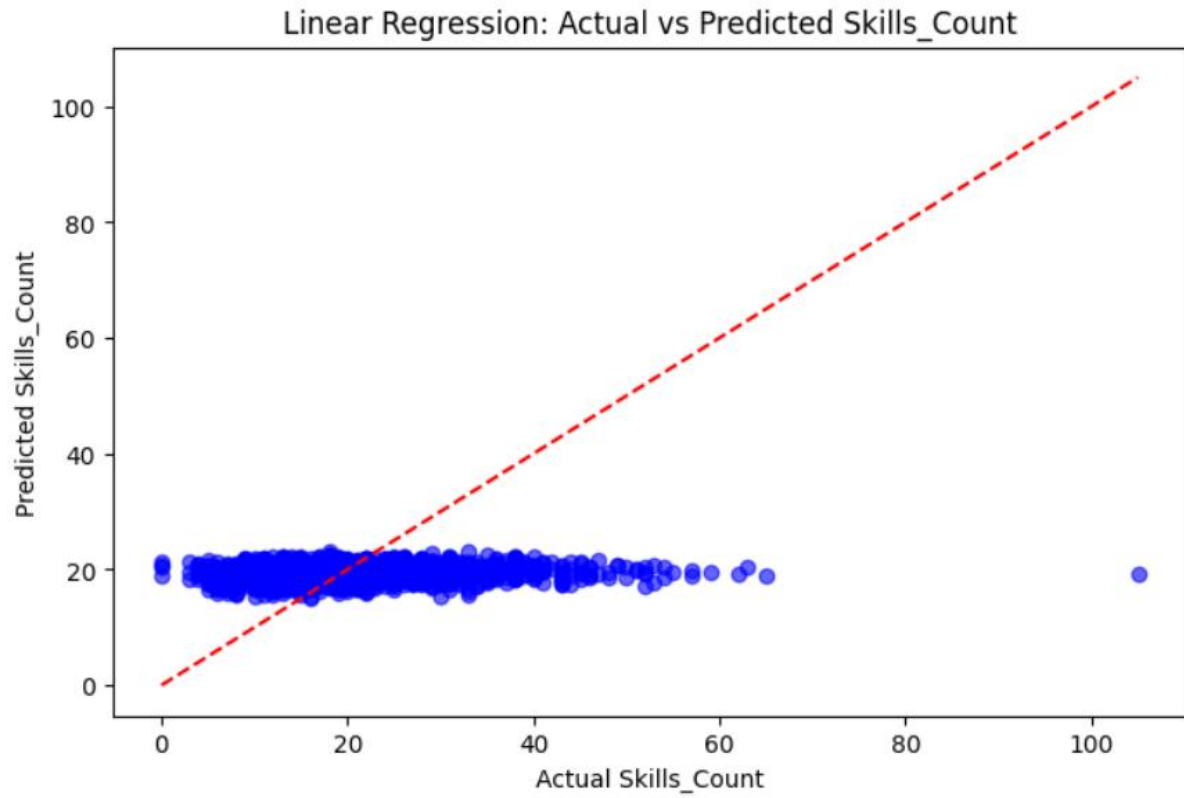
# Scale features
scaler_reg = StandardScaler()
X_train_reg = scaler_reg.fit_transform(X_train_reg)
X_test_reg = scaler_reg.transform(X_test_reg)

# Train Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg, y_train_reg)
y_pred_lin = lin_reg.predict(X_test_reg)

print(f"Linear Regression MSE: {mean_squared_error(y_test_reg, y_pred_lin):.2f}")
# Scatter plot
plt.figure(figsize=(8,5))
plt.scatter(y_test_reg, y_pred_lin, alpha=0.6, color='blue')
plt.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()], 'r--')
plt.xlabel("Actual Skills_Count")
plt.ylabel("Predicted Skills_Count")

# Residual plot
residuals = y_test_reg - y_pred_lin
plt.figure(figsize=(8,5))
plt.scatter(y_pred_lin, residuals, alpha=0.6, color='purple')
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted Skills_Count")
plt.ylabel("Residuals")
plt.title("Linear Regression Residual Plot")
plt.show()
```

Linear Regression MSE: 64.34



```

# Classification target
if 'Remote' in df.columns:
    y_clf = df['Remote']
    X_clf = df.drop(['Remote'], axis=1)
else:
    y_clf = (df['Skills_Count'] > df['Skills_Count'].median())
    X_clf = df.drop(['Skills_Count'], axis=1)

# Train-test split
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test
    X_clf, y_clf, test_size=0.2, random_state=42
)

# Scale
scaler_clf = StandardScaler()
X_train_clf = scaler_clf.fit_transform(X_train_clf)
X_test_clf = scaler_clf.transform(X_test_clf)

# Train Logistic Regression
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_clf, y_train_clf)
y_pred_log = log_reg.predict(X_test_clf)
acc_log = accuracy_score(y_test_clf, y_pred_log)
print(f"Logistic Regression Accuracy: {acc_log:.2f}")
print(classification_report(y_test_clf, y_pred_log))

```

```

# Confusion Matrix
cm_log = confusion_matrix(y_test_clf, y_pred_log)
plt.figure(figsize=(5,4))
sns.heatmap(cm_log, annot=True, fmt='d', cmap='Blues')
plt.title("Logistic Regression Confusion Matrix")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()

# ROC Curve
y_prob_log = log_reg.predict_proba(X_test_clf)[: ,1]
fpr, tpr, _ = roc_curve(y_test_clf, y_prob_log)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0,1],[0,1], 'r--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Logistic Regression ROC Curve')
plt.legend()
plt.show()

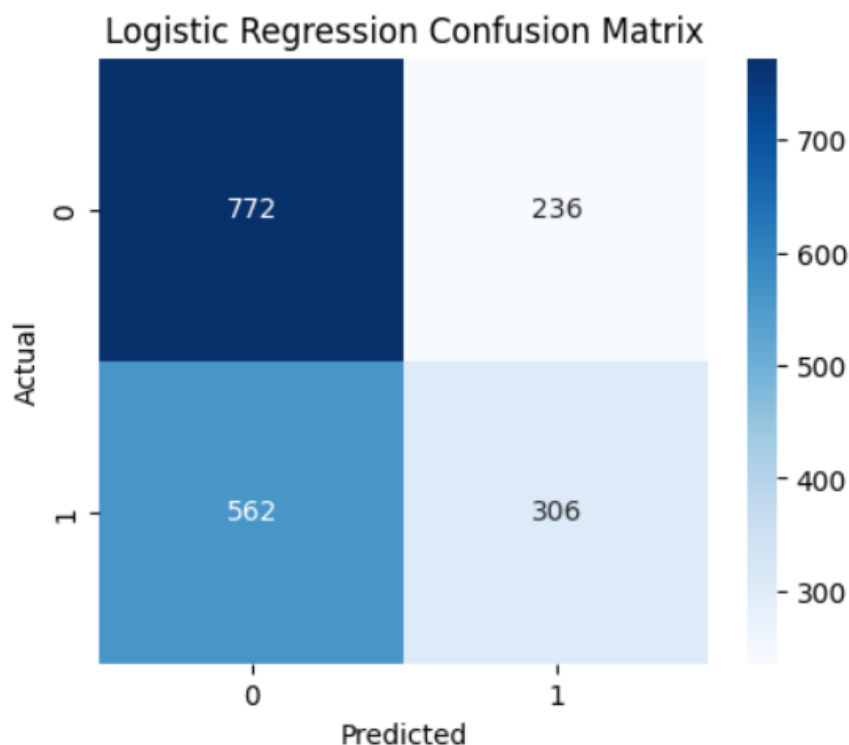
```

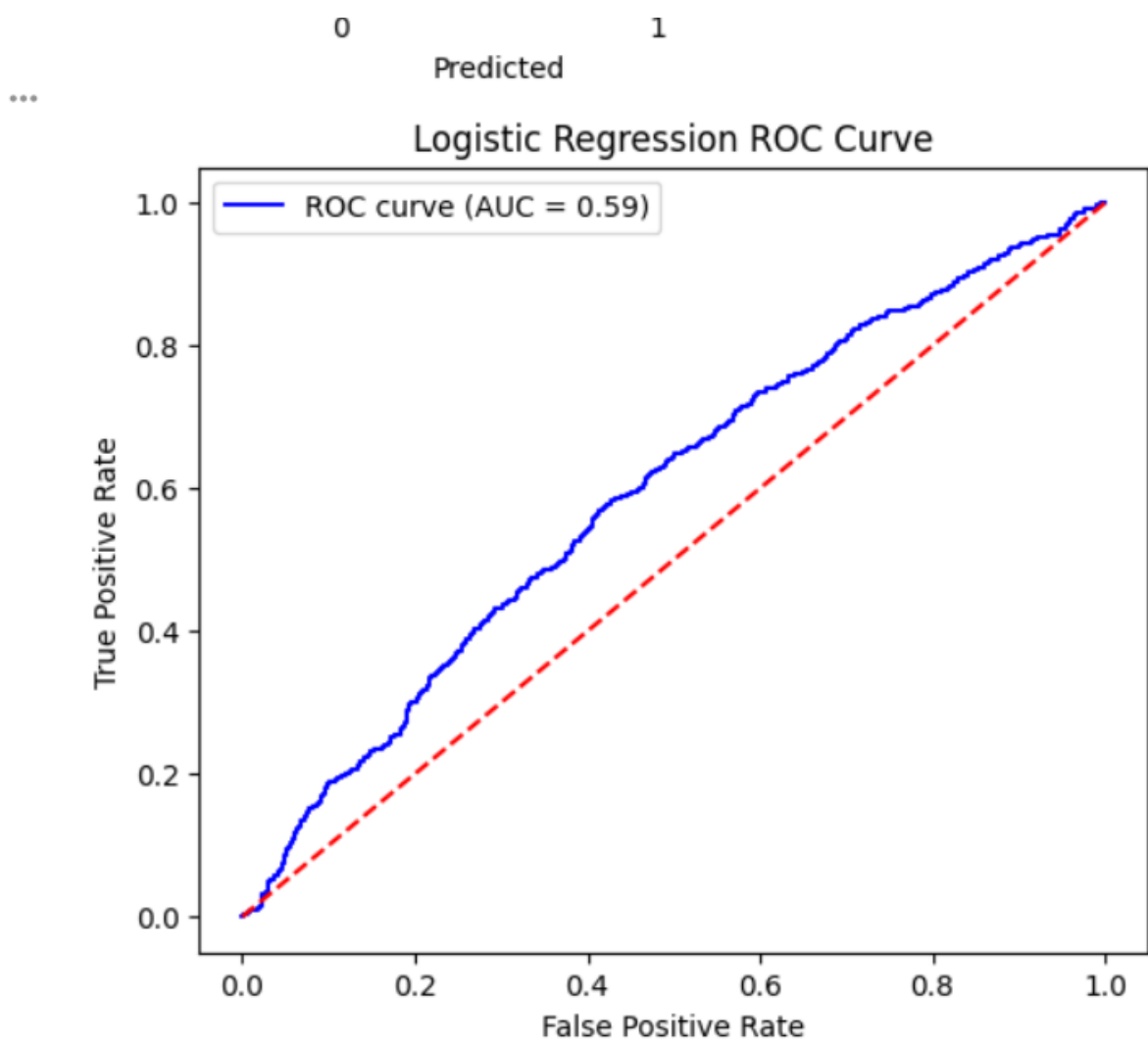
```

...          0          0.58          0.77          0.66          1008
           1          0.56          0.35          0.43           868

accuracy                                0.57          1876
macro avg          0.57          0.56          0.55          1876
weighted avg       0.57          0.57          0.56          1876

```






```

▶ # Train SVM
svm_clf = SVC(kernel='linear', probability=True)
svm_clf.fit(X_train_clf, y_train_clf)
y_pred_svm = svm_clf.predict(X_test_clf)
acc_svm = accuracy_score(y_test_clf, y_pred_svm)
print(f"SVM Accuracy: {acc_svm:.2f}")
print(classification_report(y_test_clf, y_pred_svm))

# Confusion Matrix
cm_svm = confusion_matrix(y_test_clf, y_pred_svm)
plt.figure(figsize=(5,4))
sns.heatmap(cm_svm, annot=True, fmt='d', cmap='Greens')
plt.title("SVM Confusion Matrix")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()

# ROC Curve
y_prob_svm = svm_clf.predict_proba(X_test_clf)[:,-1]
fpr_svm, tpr_svm, _ = roc_curve(y_test_clf, y_prob_svm)
roc_auc_svm = auc(fpr_svm, tpr_svm)
plt.figure(figsize=(6,5))
plt.plot(fpr_svm, tpr_svm, color='green', label=f'ROC curve (AUC = {roc_auc_svm:.2f})')
plt.plot([0,1],[0,1], 'r--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('SVM ROC Curve')
plt.legend()
plt.show()

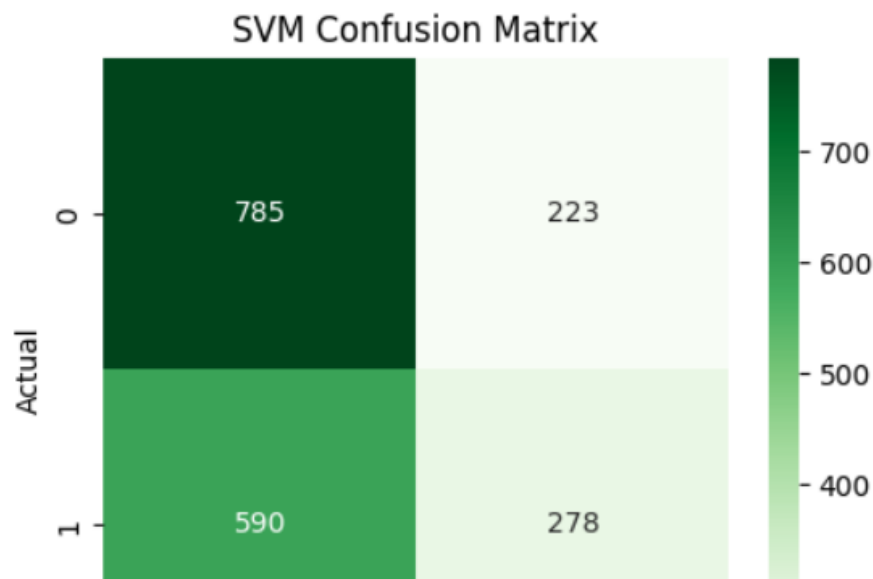
```

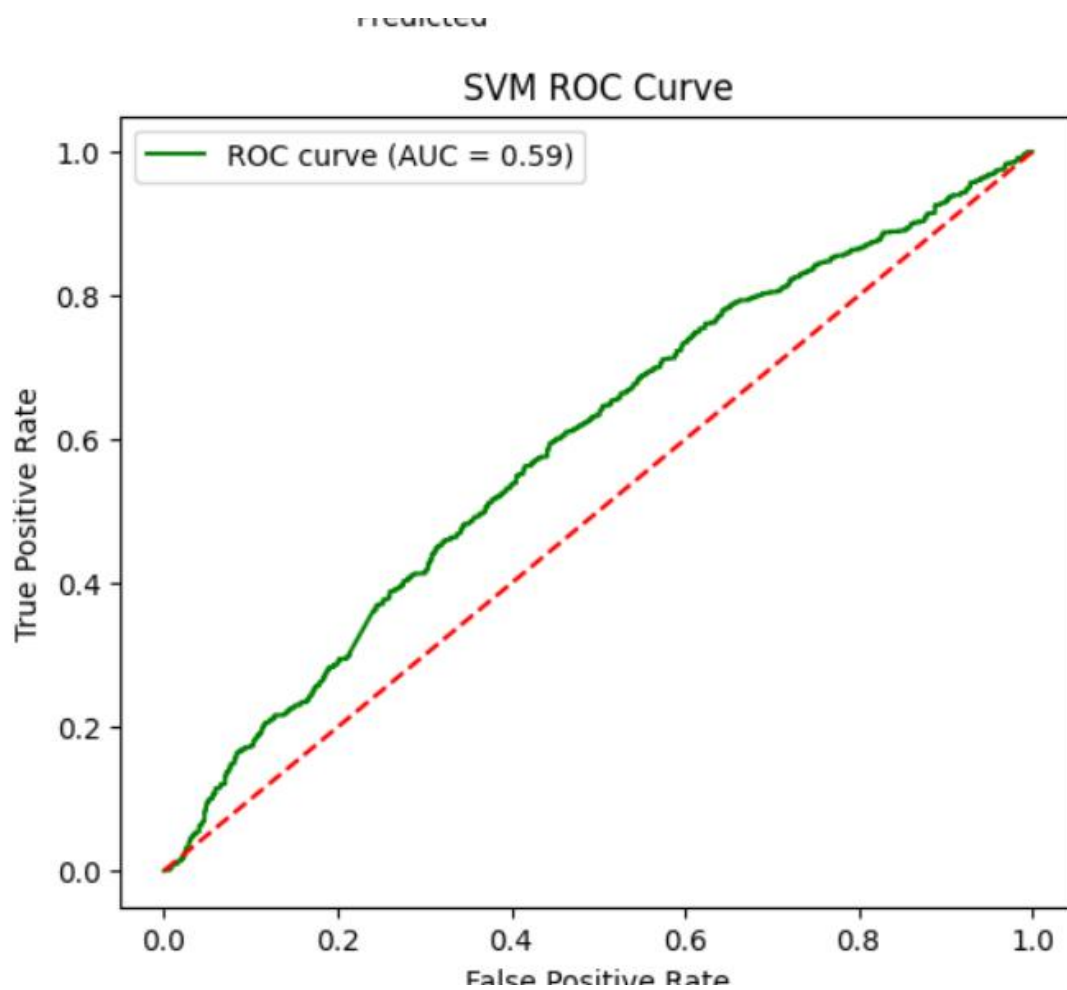
```

...      0      0.57      0.78      0.66      1008
        1      0.55      0.32      0.41      868

accuracy          0.57      1876
macro avg         0.56      1876
weighted avg      0.56      1876

```





```
models = ['Logistic Regression', 'SVM']
accuracies = [acc_log, acc_svm]

plt.figure(figsize=(6,4))
sns.barplot(x=models, y=accuracies, palette='viridis')
plt.ylim(0,1)
plt.ylabel("Accuracy")
plt.title("Classification Model Accuracy Comparison")
plt.show()
```

Classification Model Accuracy Comparison

